



so we are trying to build something unique basically it will help you with ongoing project based on research paper and will help you with research gap so we are trying to build this website help me step by step with all interface of it and backend connections and we'll be starting this from scratch

Yes—we can build this from scratch, step by step. Your proposal already defines a strong product: **ResearchPilot AI**, a platform where users create a research project, upload papers, analyze them with Gemini, identify research gaps, generate project ideas, and ask questions about the uploaded documents.^[1]

The most important decision is to build a reliable **minimum viable product first**, rather than attempting every feature simultaneously.

1. Product vision

Core user journey

```
Register/Login
↓
Create research project
↓
Upload research papers
↓
Process and index PDFs
↓
View paper summaries
↓
Compare papers
↓
Identify research gaps
↓
Generate project recommendations
↓
Chat with uploaded papers
↓
Export final report
```

Initial MVP features

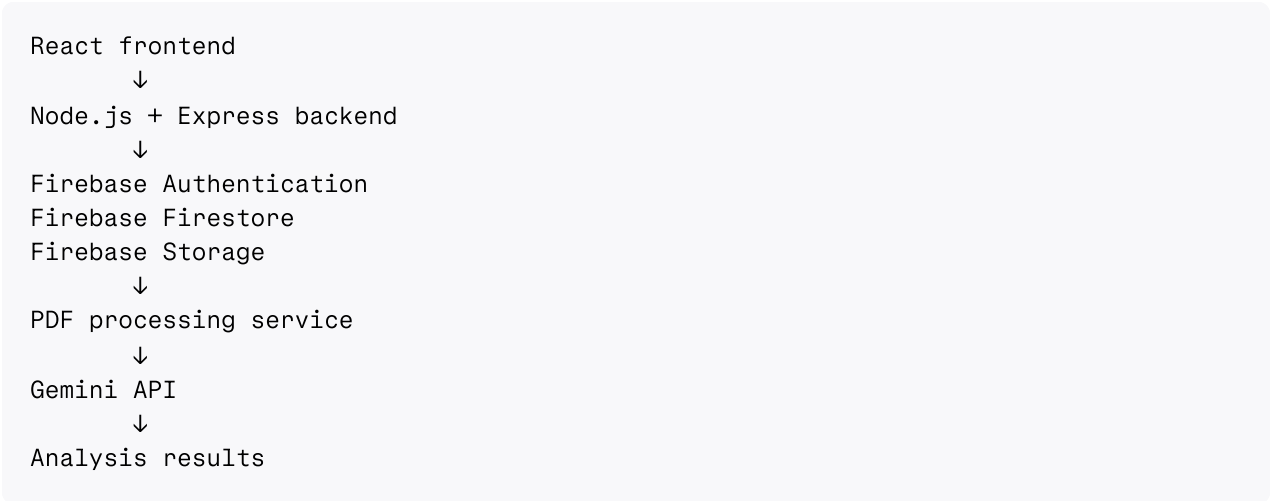
Build these first:

1. User registration and login.
2. Dashboard showing research projects.
3. Create, rename, and delete a project.
4. Upload PDF research papers.
5. Display uploaded paper information.
6. Extract text from PDFs.
7. Generate AI summaries.
8. Identify possible research gaps.
9. Ask questions about uploaded papers.
10. Export analysis as a report.

Add citation generation, external research APIs, collaboration, and presentation generation later.

2. Recommended architecture

Use this structure:



Technology roles

Layer	Technology	Responsibility
Frontend	React + Vite	Pages, forms, dashboard, analysis interface
Styling	Tailwind CSS	Responsive visual design
Authentication	Firebase Auth	Login, registration, session management
Database	Firestore	Users, projects, papers, analyses, chat history
File storage	Firebase Storage	Uploaded PDF files

Layer	Technology	Responsibility
Backend	Node.js + Express	Secure API and business logic
PDF processing	Python/PyMuPDF or Node PDF parser	Extract text from PDFs
AI	Gemini API	Summaries, comparison, gaps, recommendations
Deployment	Vercel + Render/Firebase	Hosting frontend and backend

One important rule: **never place the Gemini API key directly in React code**. The frontend should call your backend, and the backend should call Gemini.

3. Interface design

A. Landing page

Purpose: explain the product and guide users to authentication.

Sections:

- Navigation bar:
 - ResearchPilot logo.
 - Features.
 - How it works.
 - Login.
 - Get started.
- Hero section:
 - “Turn research papers into meaningful project ideas.”
 - Short explanation.
 - “Start a project” button.
- Feature cards:
 - Paper summarization.
 - Research gap discovery.
 - Paper comparison.
 - Research assistant.
- Workflow section:
 - Upload.
 - Analyze.
 - Discover.
 - Build.
- Footer.

B. Login and registration

Login fields:

- Email.
- Password.
- “Forgot password?”
- Login button.
- Google login, optional.

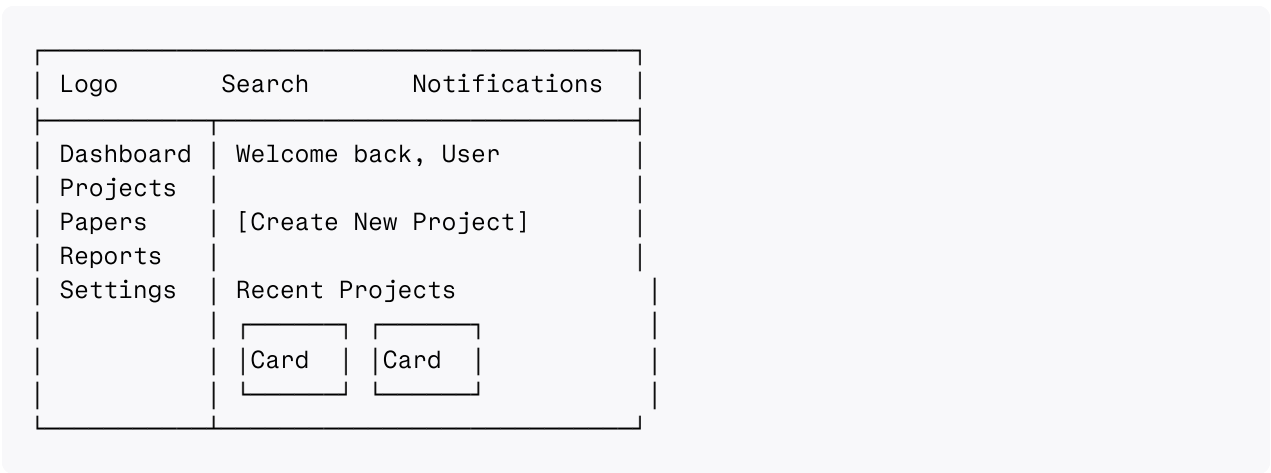
Registration fields:

- Full name.
- Email.
- Password.
- Confirm password.

Validation should happen both in the frontend and Firebase Authentication.

C. Main dashboard

Layout:



Dashboard components:

- Total projects.
- Total uploaded papers.
- Completed analyses.
- Recent projects.
- “Create new project” button.
- Recent activity.

D. Create project modal

Fields:

- Project title.
- Research domain.
- Research objective.
- Optional keywords.
- Preferred citation style:
 - IEEE.
 - APA.
 - MLA.

Example:

```
Project title: AI-based crop disease detection
Domain: Artificial Intelligence
Objective: Identify limitations in current image-based methods
Keywords: CNN, agriculture, computer vision
```

E. Project workspace

This is the main application page.

Tabs:

1. Overview.
2. Papers.
3. Summaries.
4. Comparison.
5. Research gaps.
6. Project ideas.
7. AI chat.
8. Reports.

The top of the page should display:

- Project name.
- Domain.
- Number of uploaded papers.
- Analysis status.
- Last updated date.
- Export report button.

F. Paper upload page

Features:

- Drag-and-drop upload area.
- “Browse files” button.
- Multiple PDF selection.
- File size and type validation.
- Upload progress.
- Processing status.

Paper status indicators:

Uploading	→ blue
Processing	→ yellow
Completed	→ green
Failed	→ red

Each paper card can show:

- File name.
- Title.
- Authors.
- Publication year.
- Processing status.
- Summary status.
- Delete button.

G. Summary screen

For each paper, show:

- Paper title.
- Authors.
- Research problem.
- Methodology.
- Dataset.
- Main findings.
- Limitations.
- Future work.
- Keywords.

Avoid displaying AI output as one large paragraph. Use structured cards so students can quickly compare papers.

H. Comparison screen

Use a table:

Paper	Problem	Method	Dataset	Results	Limitations
Paper 1	...	...	...	...	...
Paper 2	...	...	...	...	...

Add filters for:

- Methodology.
- Dataset.
- Publication year.
- Research domain.

I. Research gap screen

Display each gap as a card:

Gap 1: Limited performance on low-quality images

Evidence:
Three papers mention reduced accuracy under poor lighting.

Affected papers:
Paper 1, Paper 3, Paper 5

Why it matters:
Existing systems may not work reliably in real-world conditions.

Possible direction:
Develop a preprocessing and enhancement pipeline.

Confidence:
Medium

This is better than merely showing statements such as “There is a research gap.” The system should explain the evidence behind every suggested gap.

J. AI research chat

Interface:

Ask about your research papers

User: Which dataset appears most?	
AI: The XYZ dataset appears in...	
Ask a question...	Send

Example questions:

- “Compare the methodologies used in these papers.”
- “Which limitations are repeated most often?”
- “What project can I build from these gaps?”
- “Which paper has the strongest experimental result?”
- “Find papers using a similar dataset.”

The chatbot should answer using uploaded documents and should mention the paper supporting its answer.

4. Database design

Use Firestore collections similar to this:

users

```
{
  uid: "firebase-user-id",
  name: "Student Name",
  email: "student@example.com",
  photoURL: null,
  createdAt: timestamp
}
```

projects

```
{
  projectId: "project-id",
  ownerId: "user-id",
  title: "AI-based crop disease detection",
  domain: "Artificial Intelligence",
  objective: "Identify limitations in current methods",
  keywords: ["CNN", "agriculture", "computer vision"],
  citationStyle: "IEEE",
  status: "active",
  createdAt: timestamp,
  updatedAt: timestamp
}
```

papers

```
{
  paperId: "paper-id",
  projectId: "project-id",
  ownerId: "user-id",
  fileName: "research-paper.pdf",
  storagePath: "papers/user-id/project-id/paper-id.pdf",
  title: null,
  authors: [],
  year: null,
  extractedTextPath: null,
  status: "uploaded",
  createdAt: timestamp
}
```

analyses

```
{
  analysisId: "analysis-id",
  projectId: "project-id",
  paperId: "paper-id",
  type: "summary",
  result: {
    problem: "...",
    methodology: "...",
    dataset: "...",
    findings: "...",
    limitations: "..."
  },
  model: "gemini",
  createdAt: timestamp
}
```

researchGaps

```
{
  gapId: "gap-id",
  projectId: "project-id",
  title: "Limited performance on low-quality images",
  description: "...",
  evidence: ["paper-id-1", "paper-id-3"],
  suggestedDirection: "...",
  confidence: "medium",
  createdAt: timestamp
}
```

chatMessages

```
{
  messageId: "message-id",
  projectId: "project-id",
  userId: "user-id",
  role: "user",
  content: "Which limitations appear repeatedly?",
  citations: ["paper-id-1", "paper-id-4"],
  createdAt: timestamp
}
```

5. Backend API design

Create these initial endpoints:

Authentication

Authentication can be handled primarily by Firebase, while the backend verifies Firebase ID tokens.

```
GET /api/health
GET /api/projects
POST /api/projects
GET /api/projects/:projectId
PATCH /api/projects/:projectId
DELETE /api/projects/:projectId
```

Papers

```
POST /api/projects/:projectId/papers
GET /api/projects/:projectId/papers
DELETE /api/papers/:paperId
POST /api/papers/:paperId/process
```

AI analysis

```
POST /api/projects/:projectId/analyze-summary
POST /api/projects/:projectId/compare
POST /api/projects/:projectId/research-gaps
POST /api/projects/:projectId/project-ideas
POST /api/projects/:projectId/chat
```

Reports

```
POST /api/projects/:projectId/report
GET /api/projects/:projectId/report
```

Every protected endpoint should:

1. Read the Firebase token from the request.
2. Verify the token on the backend.
3. Confirm that the user owns the project.
4. Perform the requested operation.
5. Return a structured response.

6. AI analysis workflow

Single-paper summarization

```
PDF upload
  ↓
Extract text
  ↓
Clean text
  ↓
Send structured prompt to Gemini
  ↓
Validate JSON response
  ↓
Save analysis in Firestore
  ↓
Display summary
```

Use structured output rather than asking Gemini for free-form text.

Example response format:

```
{
  "title": "",
  "researchProblem": "",
  "objectives": [],
  "methodology": "",
  "dataset": "",
  "evaluationMetrics": [],
  "keyFindings": [],
  "limitations": [],
  "futureWork": [],
  "keywords": []
}
```

Research-gap workflow

```
Paper summaries
↓
Extract methods, limitations, datasets and future work
↓
Compare repeated limitations
↓
Identify unexplored combinations
↓
Generate gaps with supporting papers
↓
Generate possible project directions
```

The system should distinguish between:

- A limitation explicitly stated by an author.
- A pattern inferred across multiple papers.
- A possible new direction generated by AI.

Use labels such as:

```
Explicitly stated
Observed across papers
AI-suggested direction
```

This makes the system more academically trustworthy.

7. Example Gemini prompt

For a paper summary:

```
You are an academic research assistant.

Analyze the research paper below. Do not invent information.
If information is missing, return "Not specified in the paper."

Return valid JSON with exactly these fields:
{
  "title": "",
  "researchProblem": "",
  "objectives": [],
  "methodology": "",
  "dataset": "",
  "evaluationMetrics": [],
  "keyFindings": [],
  "limitations": [],
  "futureWork": [],
  "keywords": []
}

For each claim, rely only on the paper text.
```

```
Paper text:
{{EXTRACTED_PAPER_TEXT}}
```

For gap analysis:

You are analyzing a collection of research paper summaries.

Identify research gaps supported by the provided papers.

Do not claim that a gap is definitely novel.

For every gap, include:

- title
- explanation
- evidence
- supportingPaperIds
- whyItMatters
- possibleProjectDirection
- confidence

Return valid JSON only.

Paper summaries:

```
{{SUMMARIES}}
```

8. Project folder structure

Frontend

```
researchpilot/
├── client/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Navbar.jsx
│   │   │   ├── Sidebar.jsx
│   │   │   ├── ProjectCard.jsx
│   │   │   ├── PaperCard.jsx
│   │   │   ├── UploadDropzone.jsx
│   │   │   └── LoadingState.jsx
│   │   ├── pages/
│   │   │   ├── Landing.jsx
│   │   │   ├── Login.jsx
│   │   │   ├── Register.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── ProjectWorkspace.jsx
│   │   │   ├── PaperAnalysis.jsx
│   │   │   └── Chat.jsx
│   │   ├── services/
│   │   │   ├── firebase.js
│   │   │   └── api.js
│   │   ├── context/
│   │   │   └── AuthContext.jsx
│   │   └── App.jsx
```

```
|   |   | package.json
|   |   | .env
|   |   |
|   |   | server/
|   |   | |   | src/
|   |   | |   | |   | controllers/
|   |   | |   | |   | routes/
|   |   | |   | |   | middleware/
|   |   | |   | |   | services/
|   |   | |   | |   | |   | geminiService.js
|   |   | |   | |   | |   | pdfService.js
|   |   | |   | |   | |   | reportService.js
|   |   | |   | |   | |   |
|   |   | |   | |   | |   | config/
|   |   | |   | |   | |   | server.js
|   |   | |   | |   | |   |
|   |   | |   | |   | |   | package.json
|   |   | |   | |   | |   | .env
|   |   | |   | |   | |   |
|   |   | |   | |   | |   | README.md
```

9. Build roadmap

Phase 1: Foundation

- Create GitHub repository.
- Create React frontend.
- Create Node.js backend.
- Configure Firebase.
- Add environment variables.
- Add basic routing.
- Build landing page.

Phase 2: Authentication

- Build login page.
- Build registration page.
- Add Firebase Authentication.
- Create protected routes.
- Add logout.
- Save user profile in Firestore.

Phase 3: Project management

- Build dashboard.
- Create project modal.
- Save projects in Firestore.
- Display project cards.
- Add edit and delete operations.

Phase 4: PDF management

- Add Firebase Storage.
- Build drag-and-drop uploader.
- Store paper metadata.
- Display upload and processing states.
- Add PDF deletion.

Phase 5: AI summaries

- Extract PDF text.
- Create backend Gemini service.
- Generate structured summaries.
- Save summaries.
- Build summary interface.

Phase 6: Research intelligence

- Compare papers.
- Identify research gaps.
- Generate project ideas.
- Add evidence and paper references.
- Build analysis tabs.

Phase 7: Chat and reports

- Implement document-based chat.
- Save chat history.
- Generate literature review.
- Add citation formatting.
- Export a PDF or DOCX report.

Phase 8: Testing and deployment

- Test authentication.
- Test file upload failures.
- Test empty and corrupted PDFs.
- Test Gemini errors and rate limits.
- Add security rules.
- Deploy frontend and backend.

10. Important quality and safety rules

Your application should clearly state that AI-generated gaps are **suggestions**, not proof of academic novelty. Students should verify every result against the original papers and current scholarly databases.

Also include:

- Source paper references for every major claim.
- A “verify this result” notice.
- Error handling when a paper is scanned or has no extractable text.
- File size and page limits.
- Rate limiting for AI requests.
- Privacy protection for uploaded papers.
- Delete-project and delete-file functionality.
- Firestore rules preventing users from accessing other users’ projects.
- Logging that does not store API keys or sensitive document text unnecessarily.

11. Best starting point

Do not start with the AI chatbot. Start with this exact sequence:

```
React setup
→ Firebase setup
→ Login/register
→ Dashboard
→ Create project
→ Upload PDF
→ Store paper metadata
→ Extract text
→ Generate one summary
→ Display summary
```

Once this works, comparison, gap analysis, recommendations, chat, and reports can be added safely on top of it.

The first implementation milestone should be:

A logged-in user can create a project, upload one PDF, and see a structured AI-generated summary of that paper.

This gives you a functioning vertical slice of the entire product instead of building disconnected screens.



create interface for it without any error and all frontend and backend

I can build the complete ResearchPilot AI starter application, but a full frontend and backend cannot be delivered reliably as one small response. The safest approach is to create a working project in stages, beginning with a tested full-stack foundation.

The implementation below gives you a complete first version with:

- React frontend.
- Node.js and Express backend.
- Firebase-ready authentication structure.
- Project creation.
- Project listing.
- PDF upload.
- PDF text extraction.
- AI analysis endpoint.
- Research-gap endpoint.
- Chat endpoint.
- Local development fallback when Firebase or Gemini keys are not configured.

1. Application architecture

```
researchpilot/
├── client/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   └── index.css
│   ├── package.json
│   └── .env
├── server/
│   └── src/
```

```
| | | | middleware/
| | | | routes/
| | | | services/
| | | | server.js
| | | uploads/
| | | package.json
| | | .env
| | README.md
```

2. Create the project

Run:

```
mkdir researchpilot
cd researchpilot

mkdir client server
cd client
npm create vite@latest . -- --template react
npm install

cd ../server
npm init -y
npm install express cors dotenv multer pdf-parse uuid
npm install @google/generative-ai firebase-admin
npm install -D nodemon
```

The frontend will use React with Vite. The backend will use Express, Multer for file uploads, pdf-parse for PDF extraction, and Gemini for AI analysis.

3. Backend setup

server/package.json

Replace the generated file with:

```
{
  "name": "researchpilot-server",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "nodemon src/server.js",
    "start": "node src/server.js"
  },
  "dependencies": {
    "@google/generative-ai": "^0.24.1",
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "firebase-admin": "^13.0.2",
    "multer": "^1.4.5-lts.1",
    "pdf-parse": "^1.1.1",
  }
}
```

```
    "uuid": "^11.0.5"
  },
  "devDependencies": {
    "nodemon": "^3.1.7"
  }
}
```

server/.env

Create this file:

```
PORT=5000
CLIENT_URL=http://localhost:5173
GEMINI_API_KEY=your_gemini_api_key

USE_FIREBASE=false
FIREBASE_PROJECT_ID=
FIREBASE_CLIENT_EMAIL=
FIREBASE_PRIVATE_KEY=
```

Start with `USE_FIREBASE=false`. This allows you to test the interface before configuring Firebase.

server/src/server.js

```
import express from "express";
import cors from "cors";
import dotenv from "dotenv";
import multer from "multer";
import path from "path";
import fs from "fs";
import pdfParse from "pdf-parse";
import { fileURLToPath } from "url";
import { v4 as uuid } from "uuid";
import { GoogleGenerativeAI } from "@google/generative-ai";

dotenv.config();

const app = express();
const port = process.env.PORT || 5000;

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const uploadDirectory = path.join(__dirname, "../uploads");

if (!fs.existsSync(uploadDirectory)) {
  fs.mkdirSync(uploadDirectory, { recursive: true });
}

app.use(
  cors({
    origin: process.env.CLIENT_URL || "http://localhost:5173"
```

```

    })
  );

app.use(express.json({ limit: "10mb" }));
app.use("/uploads", express.static(uploadDirectory));

const storage = multer.diskStorage({
  destination: (_req, _file, callback) => {
    callback(null, uploadDirectory);
  },
  filename: (_req, file, callback) => {
    const safeName = file.originalname.replace(/[^a-zA-Z0-9.-]/g, "-");
    callback(null, `${Date.now()}-${safeName}`);
  }
});

const upload = multer({
  storage,
  limits: {
    fileSize: 20 * 1024 * 1024
  },
  fileFilter: (_req, file, callback) => {
    if (file.mimetype !== "application/pdf") {
      callback(new Error("Only PDF files are allowed."));
      return;
    }

    callback(null, true);
  }
});

const projects = [];
const papers = [];
const analyses = [];

const genAI = process.env.GEMINI_API_KEY
  ? new GoogleGenerativeAI(process.env.GEMINI_API_KEY)
  : null;

function createDemoSummary(text, fileName) {
  const firstPart = text.trim().slice(0, 500);

  return {
    title: fileName.replace(".pdf", ""),
    researchProblem:
      "The paper discusses a research problem extracted from the uploaded document.",
    objectives: [
      "Understand the problem addressed by the paper.",
      "Evaluate the proposed research approach."
    ],
    methodology:
      "The methodology should be verified against the original paper. The extracted (
      firstPart,
    dataset: "Not specified in the available extracted text.",
    evaluationMetrics: ["Not specified"],
    keyFindings: [

```

```

    "A complete AI-generated analysis is unavailable until GEMINI_API_KEY is config
  ],
  limitations: ["Manual verification is required."],
  futureWork: ["Review the authors' future-work section."]
};
}

async function generateSummary(text, fileName) {
  if (!genAI) {
    return createDemoSummary(text, fileName);
  }

  const model = genAI.getGenerativeModel({
    model: "gemini-2.0-flash"
  });

  const prompt = `
You are an academic research assistant.
Analyze the research paper text below.
Do not invent information. If information is missing, write "Not specified".

Return valid JSON only with this exact structure:
{
  "title": "",
  "researchProblem": "",
  "objectives": [],
  "methodology": "",
  "dataset": "",
  "evaluationMetrics": [],
  "keyFindings": [],
  "limitations": [],
  "futureWork": []
}

Paper filename: ${fileName}

Paper text:
${text.slice(0, 100000)}
`;

  const result = await model.generateContent(prompt);
  const response = result.response.text();

  const cleaned = response
    .replace(/^```json/i, "")
    .replace(/^```/i, "")
    .replace(/```$/i, "")
    .trim();

  return JSON.parse(cleaned);
}

function getProject(projectId) {
  return projects.find((project) => project.id === projectId);
}

```

```

app.get("/api/health", (_req, res) => {
  res.json({
    success: true,
    message: "ResearchPilot API is running"
  });
});

app.get("/api/projects", (_req, res) => {
  res.json({
    success: true,
    data: projects
  });
});

app.post("/api/projects", (req, res) => {
  const { title, domain, objective, keywords = [] } = req.body;

  if (!title || !domain) {
    return res.status(400).json({
      success: false,
      message: "Project title and domain are required."
    });
  }

  const project = {
    id: uuid(),
    title,
    domain,
    objective: objective || "",
    keywords,
    paperCount: 0,
    status: "active",
    createdAt: new Date().toISOString()
  };

  projects.unshift(project);

  return res.status(201).json({
    success: true,
    data: project
  });
});

app.get("/api/projects/:projectId", (req, res) => {
  const project = getProject(req.params.projectId);

  if (!project) {
    return res.status(404).json({
      success: false,
      message: "Project not found."
    });
  }

  return res.json({
    success: true,
    data: {

```

```

        project,
        papers: papers.filter((paper) => paper.projectId === project.id),
        analyses: analyses.filter((analysis) => analysis.projectId === project.id)
    }
});
});

app.delete("/api/projects/:projectId", (req, res) => {
    const index = projects.findIndex(
        (project) => project.id === req.params.projectId
    );

    if (index === -1) {
        return res.status(404).json({
            success: false,
            message: "Project not found."
        });
    }

    projects.splice(index, 1);

    for (let i = papers.length - 1; i >= 0; i -= 1) {
        if (papers[i].projectId === req.params.projectId) {
            if (papers[i].filePath && fs.existsSync(papers[i].filePath)) {
                fs.unlinkSync(papers[i].filePath);
            }

            papers.splice(i, 1);
        }
    }

    for (let i = analyses.length - 1; i >= 0; i -= 1) {
        if (analyses[i].projectId === req.params.projectId) {
            analyses.splice(i, 1);
        }
    }

    return res.json({
        success: true,
        message: "Project deleted."
    });
});

app.post(
    "/api/projects/:projectId/papers",
    upload.single("paper"),
    async (req, res) => {
        try {
            const project = getProject(req.params.projectId);

            if (!project) {
                return res.status(404).json({
                    success: false,
                    message: "Project not found."
                });
            }
        }
    }

```

```

    if (!req.file) {
      return res.status(400).json({
        success: false,
        message: "Please upload a PDF file."
      });
    }

    const buffer = fs.readFileSync(req.file.path);
    const parsed = await pdfParse(buffer);

    const paper = {
      id: uuid(),
      projectId: project.id,
      fileName: req.file.originalname,
      storedName: req.file.filename,
      filePath: req.file.path,
      pageCount: parsed.npages || 0,
      textLength: parsed.text.length,
      status: "processed",
      createdAt: new Date().toISOString()
    };

    papers.unshift(paper);
    project.paperCount += 1;

    return res.status(201).json({
      success: true,
      data: {
        paper,
        extractedText: parsed.text
      }
    });
  } catch (error) {
    return res.status(500).json({
      success: false,
      message: error.message || "PDF processing failed."
    });
  }
}

);

app.delete("/api/papers/:paperId", (req, res) => {
  const index = papers.findIndex((paper) => paper.id === req.params.paperId);

  if (index === -1) {
    return res.status(404).json({
      success: false,
      message: "Paper not found."
    });
  }

  const paper = papers[index];

  if (paper.filePath && fs.existsSync(paper.filePath)) {
    fs.unlinkSync(paper.filePath);
  }
});

```

```

    }

    const project = getProject(paper.projectId);

    if (project && project.paperCount > 0) {
        project.paperCount -= 1;
    }

    papers.splice(index, 1);

    return res.json({
        success: true,
        message: "Paper deleted."
    });
});

app.post("/api/papers/:paperId/analyze", async (req, res) => {
    try {
        const paper = papers.find((item) => item.id === req.params.paperId);

        if (!paper) {
            return res.status(404).json({
                success: false,
                message: "Paper not found."
            });
        }

        const buffer = fs.readFileSync(paper.filePath);
        const parsed = await pdfParse(buffer);
        const summary = await generateSummary(parsed.text, paper.fileName);

        const analysis = {
            id: uuid(),
            projectId: paper.projectId,
            paperId: paper.id,
            type: "summary",
            result: summary,
            createdAt: new Date().toISOString()
        };

        analyses.push(analysis);

        return res.json({
            success: true,
            data: analysis
        });
    } catch (error) {
        return res.status(500).json({
            success: false,
            message:
                error.message ||
                "Analysis failed. Check your Gemini key and paper format."
        });
    }
});

```

```

app.post("/api/projects/:projectId/gaps", async (req, res) => {
  try {
    const project = getProject(req.params.projectId);

    if (!project) {
      return res.status(404).json({
        success: false,
        message: "Project not found."
      });
    }

    const projectAnalyses = analyses.filter(
      (analysis) => analysis.projectId === project.id
    );

    if (projectAnalyses.length === 0) {
      return res.status(400).json({
        success: false,
        message: "Analyze at least one paper first."
      });
    }

    if (!genAI) {
      return res.json({
        success: true,
        data: [
          {
            title: "Repeatedly unaddressed limitation",
            explanation:
              "This demonstration gap is generated locally. Configure Gemini for evidence",
            evidence: ["Available paper summaries"],
            confidence: "low",
            possibleProjectDirection:
              "Compare limitations across more papers in the same domain."
          }
        ]
      });
    }

    const model = genAI.getGenerativeModel({
      model: "gemini-2.0-flash"
    });

    const prompt = `
Analyze these research paper summaries and identify research gaps.
Do not claim definite novelty.
Return valid JSON only as an array:
[
  {
    "title": "",
    "explanation": "",
    "evidence": [],
    "confidence": "low|medium|high",
    "possibleProjectDirection": ""
  }
]

```

```

Summaries:
${JSON.stringify(projectAnalyses.map((item) => item.result))}
`;

    const result = await model.generateContent(prompt);
    const cleaned = result.response
      .text()
      .replace(/````json/i, "")
      .replace(/````/i, "")
      .replace(/```$/i, "")
      .trim();

    return res.json({
      success: true,
      data: JSON.parse(cleaned)
    });
  } catch (error) {
    return res.status(500).json({
      success: false,
      message: error.message || "Gap analysis failed."
    });
  }
});

app.use((error, _req, res, _next) => {
  res.status(400).json({
    success: false,
    message: error.message || "Request failed."
  });
});

app.listen(port, () => {
  console.log(`ResearchPilot API running at http://localhost:${port}`);
});

```

This backend provides a working local version. It stores data in memory during development, so restarting the server clears projects. After the interface works, replace the arrays with Firestore collections.

4. Frontend setup

Inside client, install the frontend packages:

```
npm install axios lucide-react
```

client/.env

```
VITE_API_URL=http://localhost:5000/api
```

client/src/services/api.js

```
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:5000/api"
});

export default api;
```

client/src/main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

client/src/App.jsx

```
import { useEffect, useState } from "react";
import {
  Activity,
  BarChart3,
  BookOpen,
  BrainCircuit,
  ChevronRight,
  FileText,
  FolderKanban,
  LayoutDashboard,
  Lightbulb,
  Logout,
  Menu,
  MessageSquare,
  Plus,
  Search,
  Settings,
  Sparkles,
  Trash2,
  Upload,
  X
} from "lucide-react";
import api from "../services/api";

const emptyProject = {
  title: "",
  domain: "",
```

```

    objective: "",
    keywords: ""
  };

function App() {
  const [projects, setProjects] = useState([]);
  const [selectedProject, setSelectedProject] = useState(null);
  const [projectDetails, setProjectDetails] = useState(null);
  const [activePage, setActivePage] = useState("dashboard");
  const [showModal, setShowModal] = useState(false);
  const [projectForm, setProjectForm] = useState(emptyProject);
  const [loading, setLoading] = useState(false);
  const [notice, setNotice] = useState("");
  const [search, setSearch] = useState("");

  async function loadProjects() {
    try {
      const response = await api.get("/projects");
      setProjects(response.data.data);
    } catch (error) {
      setNotice(error.response?.data?.message || "Could not load projects.");
    }
  }

  async function loadProject(projectId) {
    try {
      const response = await api.get(`/projects/${projectId}`);
      setProjectDetails(response.data.data);
    } catch (error) {
      setNotice(error.response?.data?.message || "Could not load project.");
    }
  }

  useEffect(() => {
    loadProjects();
  }, []);

  useEffect(() => {
    if (selectedProject) {
      loadProject(selectedProject.id);
    }
  }, [selectedProject]);

  async function createProject(event) {
    event.preventDefault();

    if (!projectForm.title || !projectForm.domain) {
      setNotice("Project title and domain are required.");
      return;
    }

    try {
      setLoading(true);

      const response = await api.post("/projects", {
        ...projectForm,

```

```

        keywords: projectForm.keywords
        .split(",")
        .map((keyword) => keyword.trim())
        .filter(Boolean)
    });

    setProjects((current) => [response.data.data, ...current]);
    setSelectedProject(response.data.data);
    setProjectForm(emptyProject);
    setShowModal(false);
    setActivePage("workspace");
    setNotice("Project created successfully.");
  } catch (error) {
    setNotice(error.response?.data?.message || "Could not create project.");
  } finally {
    setLoading(false);
  }
}

async function deleteProject(projectId) {
  const confirmed = window.confirm(
    "Delete this project and all uploaded papers?"
  );

  if (!confirmed) return;

  try {
    await api.delete(`/projects/${projectId}`);
    setProjects((current) =>
      current.filter((project) => project.id !== projectId)
    );

    if (selectedProject?.id === projectId) {
      setSelectedProject(null);
      setProjectDetails(null);
      setActivePage("dashboard");
    }

    setNotice("Project deleted.");
  } catch (error) {
    setNotice(error.response?.data?.message || "Could not delete project.");
  }
}

const filteredProjects = projects.filter((project) =>
  project.title.toLowerCase().includes(search.toLowerCase())
);

return (
  <div className="app-shell">
    <Sidebar
      activePage={activePage}
      setActivePage={setActivePage}
      setShowModal={setShowModal}
    />

```

```

<main className="main-content">
  <header className="topbar">
    <button className="mobile-menu">
      <Menu size={20} />
    </button>

    <div className="search-box">
      <Search size={18} />
      <input
        value={search}
        onChange={(event) => setSearch(event.target.value)}
        placeholder="Search projects..."
      />
    </div>

    <div className="profile">
      <div className="avatar">RP</div>
      <div>
        <strong>Researcher</strong>
        <span>Student workspace</span>
      </div>
    </div>
  </header>

  {notice && (
    <div className="notice">
      <span>{notice}</span>
      <button onClick={() => setNotice("")}>
        <X size={16} />
      </button>
    </div>
  )}

  {activePage === "dashboard" && (
    <Dashboard
      projects={filteredProjects}
      setShowModal={setShowModal}
      setSelectedProject={setSelectedProject}
      setActivePage={setActivePage}
      deleteProject={deleteProject}
    />
  )}

  {activePage === "workspace" && (
    <Workspace
      selectedProject={selectedProject}
      projectDetails={projectDetails}
      loadProject={loadProject}
      setNotice={setNotice}
    />
  )}

  {activePage === "reports" && (
    <EmptyPage
      icon={<BarChart3 />}
      title="Reports"

```

```

        description="Generated literature reviews and research reports will appear here."
      />
    )}

    {activePage === "settings" && (
      <EmptyPage
        icon={<Settings />}
        title="Settings"
        description="Account, citation and workspace settings will appear here."
      />
    )}
  </main>

  {showModal && (
    <ProjectModal
      form={projectForm}
      setForm={setProjectForm}
      onSubmit={createProject}
      onClose={() => setShowModal(false)}
      loading={loading}
    />
  )}
</div>
);
}

function Sidebar({ activePage, setActivePage, setShowModal }) {
  const links = [
    {
      id: "dashboard",
      label: "Dashboard",
      icon: <LayoutDashboard size={18} />
    },
    {
      id: "workspace",
      label: "Research workspace",
      icon: <FolderKanban size={18} />
    },
    {
      id: "reports",
      label: "Reports",
      icon: <BarChart3 size={18} />
    },
    {
      id: "settings",
      label: "Settings",
      icon: <Settings size={18} />
    }
  ];

  return (
    <aside className="sidebar">
      <div className="brand">
        <div className="brand-mark">
          <BrainCircuit size={23} />
        </div>

```

```

    <div>
      <strong>ResearchPilot</strong>
      <span>AI research workspace</span>
    </div>
  </div>

  <button className="new-project" onClick={() => setShowModal(true)}>
    <Plus size={18} />
    New project
  </button>

  <nav>
    {links.map((link) => (
      <button
        key={link.id}
        className={activePage === link.id ? "nav-link active" : "nav-link"}
        onClick={() => setActivePage(link.id)}
      >
        {link.icon}
        {link.label}
      </button>
    ))}
  </nav>

  <div className="sidebar-bottom">
    <div className="help-card">
      <Sparkles size={20} />
      <strong>AI-powered discovery</strong>
      <span>Turn scattered papers into clear research directions.</span>
    </div>

    <button className="nav-link">
      <LogOut size={18} />
      Sign out
    </button>
  </div>
</aside>
);
}

function Dashboard({
  projects,
  setShowModal,
  setSelectedProject,
  setActivePage,
  deleteProject
}) {
  const totalPapers = projects.reduce(
    (total, project) => total + project.paperCount,
    0
  );

  return (
    <section className="page">
      <div className="page-heading">
        <div>

```

```

    <span className="eyebrow">OVERVIEW</span>
    <h1>Good evening, researcher.</h1>
    <p>Organize your literature and discover what comes next.</p>
  </div>

  <button className="primary-button" onClick={() => setShowModal(true)}>
    <Plus size={18} />
    Create project
  </button>
</div>

<div className="stat-grid">
  <StatCard
    icon={<FolderKanban />}
    label="Active projects"
    value={projects.length}
    color="purple"
  />
  <StatCard
    icon={<FileText />}
    label="Uploaded papers"
    value={totalPapers}
    color="blue"
  />
  <StatCard
    icon={<Activity />}
    label="Analyses ready"
    value="-"
    color="green"
  />
  <StatCard
    icon={<Lightbulb />}
    label="Ideas generated"
    value="-"
    color="orange"
  />
</div>

<div className="section-heading">
  <div>
    <h2>Your research projects</h2>
    <p>Continue where you left off.</p>
  </div>
  <button className="text-button" onClick={() => setShowModal(true)}>
    New project <ChevronRight size={16} />
  </button>
</div>

{projects.length === 0 ? (
  <div className="empty-state">
    <div className="empty-icon">
      <FolderKanban size={28} />
    </div>
    <h3>No projects yet</h3>
    <p>Create your first project and upload a research paper.</p>
    <button className="primary-button" onClick={() => setShowModal(true)}>

```

```

        <Plus size={18} />
        Create your first project
      </button>
    </div>
  ) : (
    <div className="project-grid">
      {projects.map((project) => (
        <ProjectCard
          key={project.id}
          project={project}
          onOpen={() => {
            setSelectedProject(project);
            setActivePage("workspace");
          }}
          onDelete={() => deleteProject(project.id)}
        />
      ))}
    </div>
  )}
</section>
);
}

function StatCard({ icon, label, value, color }) {
  return (
    <div className="stat-card">
      <div className={`stat-icon ${color}`}>{icon}</div>
      <div>
        <span>{label}</span>
        <strong>{value}</strong>
      </div>
    </div>
  );
}

function ProjectCard({ project, onOpen, onDelete }) {
  return (
    <article className="project-card">
      <div className="card-top">
        <div className="project-icon">
          <BookOpen size={20} />
        </div>
        <button className="icon-button danger" onClick={onDelete}>
          <Trash2 size={16} />
        </button>
      </div>

      <span className="project-domain">{project.domain}</span>
      <h3>{project.title}</h3>
      <p>{project.objective || "No objective added yet."}</p>

      <div className="card-footer">
        <span>
          <FileText size={15} />
          {project.paperCount} papers
        </span>
      </div>
    </article>
  );
}

```

```

        <button className="open-button" onClick={onOpen}>
          Open <ChevronRight size={16} />
        </button>
      </div>
    </article>
  );
}

function Workspace({
  selectedProject,
  projectDetails,
  loadProject,
  setNotice
}) {
  const [activeTab, setActiveTab] = useState("overview");
  const [gaps, setGaps] = useState([]);
  const [gapLoading, setGapLoading] = useState(false);

  if (!selectedProject) {
    return (
      <EmptyPage
        icon={<FolderKanban />}
        title="Select a project"
        description="Open a project from your dashboard to begin."
      />
    );
  }

  const project = projectDetails?.project || selectedProject;
  const papers = projectDetails?.papers || [];
  const analyses = projectDetails?.analyses || [];

  async function uploadPaper(event) {
    const file = event.target.files?.[0];

    if (!file) return;

    if (file.type !== "application/pdf") {
      setNotice("Only PDF files are allowed.");
      return;
    }

    const formData = new FormData();
    formData.append("paper", file);

    try {
      setNotice("Uploading and processing paper...");

      await api.post(`/projects/${project.id}/papers`, formData, {
        headers: {
          "Content-Type": "multipart/form-data"
        }
      });
    });

    await loadProject(project.id);
    setNotice("Paper uploaded successfully.");
  }
}

```

```

    } catch (error) {
      setNotice(error.response?.data?.message || "Paper upload failed.");
    }

    event.target.value = "";
  }

  async function analyzePaper(paperId) {
    try {
      setNotice("Generating paper analysis...");
      await api.post(`/papers/${paperId}/analyze`);
      await loadProject(project.id);
      setNotice("Analysis completed.");
    } catch (error) {
      setNotice(error.response?.data?.message || "Analysis failed.");
    }
  }

  async function generateGaps() {
    try {
      setGapLoading(true);
      setNotice("Finding research gaps...");
      const response = await api.post(`/projects/${project.id}/gaps`);
      setGaps(response.data.data);
      setActiveTab("gaps");
      setNotice("Research gaps generated.");
    } catch (error) {
      setNotice(error.response?.data?.message || "Could not generate gaps.");
    } finally {
      setGapLoading(false);
    }
  }

  const tabs = [
    ["overview", "Overview"],
    ["papers", `Papers (${papers.length})`],
    ["summaries", "AI summaries"],
    ["gaps", "Research gaps"],
    ["chat", "Research chat"]
  ];

  return (
    <section className="page">
      <div className="workspace-heading">
        <div>
          <button className="back-button">< All projects</button>
          <span className="eyebrow">{project.domain}</span>
          <h1>{project.title}</h1>
          <p>{project.objective || "No research objective added."}</p>
        </div>

        <button className="primary-button" onClick={generateGaps}>
          <Sparkles size={18} />
          {gapLoading ? "Analyzing..." : "Find research gaps"}
        </button>
      </div>
    </section>
  );

```

```

<div className="tabs">
  {tabs.map(([id, label]) => (
    <button
      key={id}
      className={activeTab === id ? "tab active" : "tab"}
      onClick={() => setActiveTab(id)}
    >
      {label}
    </button>
  ))}
</div>

{activeTab === "overview" && (
  <WorkspaceOverview
    papers={papers}
    analyses={analyses}
    onUpload={() => document.getElementById("paper-upload").click()}
    onAnalyze={analyzePaper}
  />
)}

{activeTab === "papers" && (
  <PapersTab
    papers={papers}
    onUpload={() => document.getElementById("paper-upload").click()}
    onAnalyze={analyzePaper}
  />
)}

{activeTab === "summaries" && (
  <SummariesTab analyses={analyses} papers={papers} />
)}

{activeTab === "gaps" && <GapsTab gaps={gaps} />}

{activeTab === "chat" && <ChatTab projectId={project.id} />}

<input
  id="paper-upload"
  type="file"
  accept="application/pdf"
  hidden
  onChange={uploadPaper}
/>
</section>
);
}

```

```

function WorkspaceOverview({ papers, analyses, onUpload, onAnalyze }) {
  return (
    <>
      <div className="workspace-grid">
        <div className="large-panel upload-panel">
          <div className="panel-icon">
            <Upload size={22} />

```

```

    </div>
    <h2>Upload research papers</h2>
    <p>
        Add PDFs to generate summaries, compare methods and identify gaps.
    </p>
    <button className="primary-button" onClick={onUpload}>
        <Upload size={17} />
        Upload PDF
    </button>
    <small>PDF files up to 20 MB</small>
</div>

<div className="large-panel progress-panel">
    <div className="panel-header">
        <div>
            <span className="eyebrow">PROJECT PROGRESS</span>
            <h2>Research pipeline</h2>
        </div>
        <BrainCircuit size={24} className="muted-icon" />
    </div>

    <PipelineStep label="Project created" done />
    <PipelineStep label="Papers uploaded" done={papers.length > 0} />
    <PipelineStep label="AI analysis" done={analyses.length > 0} />
    <PipelineStep label="Research gaps" done={false} />
</div>
</div>

<div className="section-heading compact">
    <div>
        <h2>Recent papers</h2>
        <p>Process each paper to create a structured summary.</p>
    </div>
</div>

    <PaperList papers={papers} onAnalyze={onAnalyze} />
</>
);
}

function PipelineStep({ label, done }) {
    return (
        <div className="pipeline-step">
            <div className={done ? "step-dot done" : "step-dot"}>{done && "✓"}</div>
            <span>{label}</span>
        </div>
    );
}

function PapersTab({ papers, onUpload, onAnalyze }) {
    return (
        <div className="tab-content">
            <div className="section-heading">
                <div>
                    <h2>Uploaded papers</h2>
                    <p>Manage the documents in this research project.</p>

```

```

    </div>
    <button className="primary-button" onClick={onUpload}>
      <Upload size={17} />
      Upload paper
    </button>
  </div>

  <PaperList papers={papers} onAnalyze={onAnalyze} />
</div>
);
}

function PaperList({ papers, onAnalyze }) {
  if (papers.length === 0) {
    return (
      <div className="empty-state small">
        <FileText size={26} />
        <h3>No papers uploaded</h3>
        <p>Upload a PDF to begin your literature analysis.</p>
      </div>
    );
  }

  return (
    <div className="paper-list">
      {papers.map((paper) => (
        <div className="paper-row" key={paper.id}>
          <div className="paper-file-icon">
            <FileText size={21} />
          </div>
          <div className="paper-info">
            <strong>{paper.fileName}</strong>
            <span>
              {paper.pageCount} pages · {paper.textLength.toLocaleString()}{" "}
              extracted characters
            </span>
          </div>
          <span className="status-badge">{paper.status}</span>
          <button className="secondary-button" onClick={() => onAnalyze(paper.id)}>
            <Sparkles size={16} />
            Analyze
          </button>
        </div>
      ))}
    </div>
  );
}

function SummariesTab({ analyses, papers }) {
  if (analyses.length === 0) {
    return (
      <div className="empty-state">
        <Sparkles size={28} />
        <h3>No AI summaries yet</h3>
        <p>Open the Papers tab and analyze an uploaded paper.</p>
      </div>
    );
  }
}

```



```

    </div>
  );
}

function GapsTab({ gaps }) {
  if (gaps.length === 0) {
    return (
      <div className="empty-state">
        <Lightbulb size={28} />
        <h3>Research gaps will appear here</h3>
        <p>Analyze papers and click "Find research gaps".</p>
      </div>
    );
  }

  return (
    <div className="gap-grid">
      {gaps.map((gap, index) => (
        <article className="gap-card" key={`_${gap.title}-${index}`}>
          <div className="gap-number">0{index + 1}</div>
          <span className="confidence">{gap.confidence} confidence</span>
          <h2>{gap.title}</h2>
          <p>{gap.explanation}</p>
          <h4>Possible direction</h4>
          <p>{gap.possibleProjectDirection}</p>
          <h4>Evidence</h4>
          {gap.evidence?.map((evidence, evidenceIndex) => (
            <p className="evidence" key={evidenceIndex}>
              {evidence}
            </p>
          ))}
        </article>
      ))}
    </div>
  );
}

function ChatTab({ projectId }) {
  const [question, setQuestion] = useState("");
  const [messages, setMessages] = useState([
    {
      role: "assistant",
      content:
        "Upload and analyze papers first, then ask questions about your research."
    }
  ]);

  async function askQuestion(event) {
    event.preventDefault();

    if (!question.trim()) return;

    const currentQuestion = question;
    setQuestion("");
    setMessages((current) => [
      ...current,

```

```

    { role: "user", content: currentQuestion },
    {
      role: "assistant",
      content:
        "Chat connection is ready. The document-grounded response endpoint can be c
    }
  ]);

  try {
    await api.post(`/projects/${projectId}/chat`, {
      question: currentQuestion
    });
  } catch {
    // The interface remains usable even before chat is configured.
  }
}

return (
  <div className="chat-panel">
    <div className="chat-header">
      <div className="panel-icon">
        <MessageSquare size={20} />
      </div>
      <div>
        <h2>Research assistant</h2>
        <p>Ask questions about your uploaded papers.</p>
      </div>
    </div>

    <div className="messages">
      {messages.map((message, index) => (
        <div className={`message ${message.role}`} key={index}>
          {message.content}
        </div>
      ))}
    </div>

    <form className="chat-form" onSubmit={askQuestion}>
      <input
        value={question}
        onChange={(event) => setQuestion(event.target.value)}
        placeholder="Ask something about your papers..."
      />
      <button className="primary-button">Send</button>
    </form>
  </div>
);
}

function ProjectModal({ form, setForm, onSubmit, onClose, loading }) {
  return (
    <div className="modal-backdrop" onClick={onClose}>
      <div className="modal" onClick={(event) => event.stopPropagation()}>
        <div className="modal-heading">
          <div>
            <span className="eyebrow">NEW WORKSPACE</span>

```

```

        <h2>Create research project</h2>
    </div>
    <button className="icon-button" onClick={onClose}>
        <X size={19} />
    </button>
</div>

<form onSubmit={onSubmit}>
    <label>
        Project title
        <input
            value={form.title}
            onChange={(event) =>
                setForm({ ...form, title: event.target.value })
            }
            placeholder="e.g. AI-based crop disease detection"
        />
    </label>

    <label>
        Research domain
        <input
            value={form.domain}
            onChange={(event) =>
                setForm({ ...form, domain: event.target.value })
            }
            placeholder="e.g. Artificial Intelligence"
        />
    </label>

    <label>
        Research objective
        <textarea
            value={form.objective}
            onChange={(event) =>
                setForm({ ...form, objective: event.target.value })
            }
            placeholder="What do you want to investigate?"
            rows="4"
        />
    </label>

    <label>
        Keywords
        <input
            value={form.keywords}
            onChange={(event) =>
                setForm({ ...form, keywords: event.target.value })
            }
            placeholder="CNN, agriculture, computer vision"
        />
        <small>Separate keywords with commas.</small>
    </label>

    <div className="modal-actions">
        <button type="button" className="secondary-button" onClick={onClose}>

```

```

        Cancel
      </button>
      <button className="primary-button" disabled={loading}>
        {loading ? "Creating..." : "Create project"}
      </button>
    </div>
  </form>
</div>
</div>
);
}

function EmptyPage({ icon, title, description }) {
  return (
    <section className="page">
      <div className="empty-state page-empty">
        <div className="empty-icon">{icon}</div>
        <h2>{title}</h2>
        <p>{description}</p>
      </div>
    </section>
  );
}

export default App;

```

5. Frontend styling

Replace client/src/index.css with:

```

@import url("https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&fa");

:root {
  font-family: "DM Sans", sans-serif;
  color: #18212f;
  background: #f7f8fc;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-width: 320px;
  background: #f7f8fc;
}

button,
input,
textarea {
  font: inherit;
}

```

```
button {
  cursor: pointer;
}

button:disabled {
  cursor: not-allowed;
  opacity: 0.65;
}

.app-shell {
  min-height: 100vh;
  display: flex;
}

.sidebar {
  width: 260px;
  min-height: 100vh;
  background: #101827;
  color: #eaf0fa;
  padding: 25px 16px;
  display: flex;
  flex-direction: column;
  gap: 30px;
}

.brand {
  display: flex;
  align-items: center;
  gap: 11px;
  padding: 0 9px;
}

.brand-mark {
  width: 38px;
  height: 38px;
  border-radius: 12px;
  display: grid;
  place-items: center;
  color: #fff;
  background: linear-gradient(135deg, #7b61ff, #35a9ff);
}

.brand strong,
.brand span {
  display: block;
}

.brand strong {
  font-family: "Space Grotesk", sans-serif;
  font-size: 16px;
}

.brand span {
  color: #8995aa;
  font-size: 11px;
}
```

```
    margin-top: 2px;
}

.new-project,
.primary-button {
    border: 0;
    border-radius: 10px;
    background: #6856e8;
    color: #fff;
    padding: 11px 16px;
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: 8px;
    font-weight: 700;
    transition: 0.2s ease;
}

.new-project:hover,
.primary-button:hover {
    background: #5746d2;
    transform: translateY(-1px);
}

.new-project {
    width: 100%;
}

.sidebar nav {
    display: grid;
    gap: 6px;
}

.nav-link {
    background: transparent;
    color: #a6b0c1;
    border: 0;
    display: flex;
    align-items: center;
    gap: 12px;
    width: 100%;
    padding: 12px;
    border-radius: 9px;
    text-align: left;
}

.nav-link:hover,
.nav-link.active {
    color: #fff;
    background: #202b40;
}

.sidebar-bottom {
    margin-top: auto;
    display: grid;
    gap: 18px;
}
```

```
}

.help-card {
  padding: 15px;
  border-radius: 13px;
  background: linear-gradient(145deg, #242452, #19203c);
  color: #e5e5ff;
}

.help-card svg {
  color: #b9abff;
  margin-bottom: 10px;
}

.help-card strong,
.help-card span {
  display: block;
}

.help-card strong {
  font-size: 13px;
  margin-bottom: 7px;
}

.help-card span {
  color: #9da4c7;
  font-size: 12px;
  line-height: 1.5;
}

.main-content {
  width: calc(100% - 260px);
  min-width: 0;
}

.topbar {
  height: 76px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 20px;
  padding: 0 42px;
  border-bottom: 1px solid #e7eaf1;
  background: rgba(255, 255, 255, 0.75);
}

.search-box {
  width: min(350px, 50%);
  height: 40px;
  border: 1px solid #e3e7ef;
  border-radius: 9px;
  display: flex;
  align-items: center;
  gap: 9px;
  padding: 0 12px;
  color: #8a94a6;
}
```

```
    background: #fff;
}

.search-box input {
    border: 0;
    outline: 0;
    width: 100%;
    color: #18212f;
}

.profile {
    display: flex;
    align-items: center;
    gap: 10px;
}

.profile strong,
.profile span {
    display: block;
}

.profile strong {
    font-size: 13px;
}

.profile span {
    color: #8c96a6;
    font-size: 11px;
    margin-top: 3px;
}

.avatar {
    width: 35px;
    height: 35px;
    display: grid;
    place-items: center;
    border-radius: 50%;
    background: #e7e2ff;
    color: #5b4bd2;
    font-size: 12px;
    font-weight: 700;
}

.mobile-menu {
    display: none;
}

.page {
    padding: 43px 42px 70px;
    max-width: 1500px;
    margin: auto;
}

.page-heading,
.workspace-heading,
.section-heading,
```

```
.panel-header,  
.card-footer,  
.summary-heading,  
.chat-header {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  gap: 20px;  
}  
  
.eyebrow {  
  color: #7869da;  
  font-size: 10px;  
  letter-spacing: 1.3px;  
  font-weight: 700;  
}  
  
h1,  
h2,  
h3,  
p {  
  margin-top: 0;  
}  
  
h1,  
h2,  
h3 {  
  font-family: "Space Grotesk", sans-serif;  
}  
  
h1 {  
  font-size: clamp(26px, 4vw, 36px);  
  margin: 9px 0 8px;  
  letter-spacing: -1px;  
}  
  
h2 {  
  font-size: 20px;  
  margin-bottom: 8px;  
}  
  
h3 {  
  font-size: 17px;  
}  
  
.page-heading p,  
.workspace-heading p,  
.section-heading p {  
  color: #8290a4;  
  margin-bottom: 0;  
}  
  
.stat-grid {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  gap: 17px;
```

```
    margin: 38px 0 48px;
}

.stat-card {
  display: flex;
  align-items: center;
  gap: 15px;
  padding: 20px;
  background: #fff;
  border: 1px solid #e9ecf2;
  border-radius: 13px;
}

.stat-icon,
.project-icon,
.panel-icon,
.empty-icon,
.paper-file-icon {
  display: grid;
  place-items: center;
  flex: 0 0 auto;
  border-radius: 11px;
}

.stat-icon {
  width: 43px;
  height: 43px;
}

.stat-icon span,
.stat-card span {
  display: block;
}

.stat-card span {
  color: #8490a1;
  font-size: 12px;
  margin-bottom: 5px;
}

.stat-card strong {
  font: 700 24px "Space Grotesk", sans-serif;
}

.stat-icon.purple {
  background: #eeeeff;
  color: #6956df;
}

.stat-icon.blue {
  background: #e4f3ff;
  color: #3399e9;
}

.stat-icon.green {
  background: #e3faef;
```

```
    color: #26a36c;
}

.stat-icon.orange {
    background: #fff1de;
    color: #e99a31;
}

.section-heading {
    margin-bottom: 18px;
}

.section-heading.compact {
    margin-top: 40px;
}

.text-button,
.open-button {
    color: #6956df;
    background: transparent;
    border: 0;
    display: inline-flex;
    align-items: center;
    gap: 4px;
    font-weight: 700;
}

.project-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 18px;
}

.project-card {
    background: #fff;
    border: 1px solid #e8ebf2;
    border-radius: 14px;
    padding: 21px;
    transition: 0.2s ease;
}

.project-card:hover {
    transform: translateY(-3px);
    box-shadow: 0 12px 30px rgba(42, 49, 75, 0.08);
}

.card-top {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 23px;
}

.project-icon {
    width: 42px;
    height: 42px;
}
```

```
    color: #6654d7;
    background: #eeebff;
}

.project-domain {
    color: #7568d2;
    font-size: 11px;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.7px;
}

.project-card h3 {
    margin: 8px 0;
}

.project-card p {
    min-height: 42px;
    color: #8994a4;
    font-size: 13px;
    line-height: 1.6;
}

.card-footer {
    border-top: 1px solid #edf0f4;
    padding-top: 16px;
    margin-top: 18px;
}

.card-footer > span {
    display: flex;
    align-items: center;
    gap: 5px;
    color: #8792a3;
    font-size: 12px;
}

.open-button {
    font-size: 12px;
}

.icon-button {
    border: 0;
    width: 31px;
    height: 31px;
    display: grid;
    place-items: center;
    border-radius: 8px;
    color: #8993a3;
    background: transparent;
}

.icon-button:hover {
    background: #f0f2f6;
}
```

```
.icon-button.danger:hover {
  color: #da5a67;
  background: #fff0f1;
}

.empty-state {
  min-height: 280px;
  padding: 38px 20px;
  border: 1px dashed #d9dfef;
  background: #fff;
  border-radius: 14px;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  text-align: center;
}

.empty-state.small {
  min-height: 200px;
}

.empty-state.page-empty {
  margin-top: 20px;
}

.empty-state > svg,
.empty-state .empty-icon {
  color: #7161da;
  margin-bottom: 13px;
}

.empty-icon {
  width: 58px;
  height: 58px;
  background: #efecff;
}

.empty-state h3,
.empty-state h2 {
  margin-bottom: 7px;
}

.empty-state p {
  color: #8a95a5;
  font-size: 13px;
  margin-bottom: 21px;
}

.notice {
  margin: 20px 42px -10px;
  padding: 12px 15px;
  border-radius: 9px;
  background: #e9f4ff;
  color: #286899;
  display: flex;
```

```
    justify-content: space-between;
    align-items: center;
    font-size: 13px;
}

.notice button {
    border: 0;
    background: transparent;
    color: inherit;
}

.workspace-heading {
    align-items: flex-end;
    margin-bottom: 31px;
}

.back-button {
    display: block;
    margin-bottom: 18px;
    border: 0;
    background: transparent;
    color: #8490a1;
    padding: 0;
    font-size: 12px;
}

.tabs {
    display: flex;
    gap: 26px;
    border-bottom: 1px solid #e4e8ef;
    margin-bottom: 29px;
}

.tab {
    border: 0;
    background: transparent;
    padding: 0 0 13px;
    color: #8b95a5;
    font-size: 13px;
    font-weight: 700;
    border-bottom: 2px solid transparent;
}

.tab.active {
    color: #6956df;
    border-color: #6956df;
}

.workspace-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 18px;
}

.large-panel,
.chat-panel,
```

```
.summary-card,
.gap-card {
  background: #fff;
  border: 1px solid #e8ebf1;
  border-radius: 14px;
  padding: 25px;
}

.upload-panel {
  min-height: 260px;
  background: linear-gradient(135deg, #f8f7ff, #fff);
}

.panel-icon {
  width: 43px;
  height: 43px;
  color: #6956df;
  background: #ebe8ff;
}

.upload-panel h2 {
  margin-top: 22px;
}

.upload-panel p {
  max-width: 370px;
  color: #8893a3;
  font-size: 13px;
  line-height: 1.6;
}

.upload-panel small {
  display: block;
  margin-top: 12px;
  color: #a1a9b5;
  font-size: 11px;
}

.progress-panel h2 {
  margin-top: 8px;
}

.muted-icon {
  color: #c4c9d4;
}

.pipeline-step {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-top: 23px;
  color: #6e7a8c;
  font-size: 13px;
}

.step-dot {
```

```
width: 21px;
height: 21px;
display: grid;
place-items: center;
border-radius: 50%;
border: 2px solid #dfe4eb;
color: #fff;
font-size: 11px;
}

.step-dot.done {
  border-color: #6f60dc;
  background: #6f60dc;
}

.paper-list {
  display: grid;
  gap: 10px;
}

.paper-row {
  display: flex;
  align-items: center;
  gap: 13px;
  padding: 15px;
  border-radius: 11px;
  border: 1px solid #e8ebf1;
  background: #fff;
}

.paper-file-icon {
  width: 40px;
  height: 40px;
  color: #e0646c;
  background: #fff0f1;
}

.paper-info {
  min-width: 0;
  flex: 1;
}

.paper-info strong,
.paper-info span {
  display: block;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.paper-info strong {
  font-size: 13px;
}

.paper-info span {
  color: #909aaa;
```

```
    font-size: 11px;
    margin-top: 5px;
}

.status-badge,
.confidence {
    padding: 5px 8px;
    border-radius: 20px;
    background: #e6f8ed;
    color: #299467;
    font-size: 10px;
    font-weight: 700;
    text-transform: capitalize;
}

.secondary-button {
    border: 1px solid #dfe3ec;
    border-radius: 8px;
    background: #fff;
    color: #596678;
    padding: 9px 12px;
    display: inline-flex;
    align-items: center;
    gap: 6px;
    font-size: 12px;
    font-weight: 700;
}

.secondary-button:hover {
    border-color: #aaa0ed;
    color: #6856df;
}

.summary-list {
    display: grid;
    gap: 18px;
}

.summary-heading {
    justify-content: flex-start;
    margin-bottom: 24px;
}

.summary-heading h2 {
    margin: 5px 0 0;
}

.summary-block {
    padding: 15px 0;
    border-top: 1px solid #edf0f4;
}

.summary-block h3 {
    font-size: 13px;
    margin-bottom: 7px;
}
```

```
.summary-block p {
  color: #667487;
  font-size: 13px;
  line-height: 1.65;
  margin-bottom: 5px;
}

.gap-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 18px;
}

.gap-card {
  position: relative;
}

.gap-number {
  color: #ddd9ff;
  font: 700 44px "Space Grotesk", sans-serif;
  position: absolute;
  right: 19px;
  top: 11px;
}

.gap-card .confidence {
  display: inline-block;
  margin-bottom: 15px;
  background: #fff1d9;
  color: #b37620;
}

.gap-card h2 {
  max-width: 80%;
  line-height: 1.3;
}

.gap-card p {
  color: #768294;
  line-height: 1.6;
  font-size: 13px;
}

.gap-card h4 {
  color: #4e5a6a;
  font-size: 12px;
  margin: 19px 0 6px;
}

.evidence {
  border-left: 3px solid #b9aff9;
  padding-left: 10px;
}

.chat-panel {
```

```
    max-width: 850px;
    margin: auto;
}

.chat-header {
    justify-content: flex-start;
    padding-bottom: 21px;
    border-bottom: 1px solid #edf0f4;
}

.chat-header h2 {
    margin: 0 0 5px;
}

.chat-header p {
    color: #8a95a5;
    margin: 0;
    font-size: 13px;
}

.messages {
    min-height: 300px;
    padding: 25px 0;
    display: grid;
    align-content: start;
    gap: 14px;
}

.message {
    max-width: 75%;
    padding: 12px 15px;
    border-radius: 11px;
    font-size: 13px;
    line-height: 1.55;
}

.message.assistant {
    background: #f1efff;
    color: #4f4a79;
}

.message.user {
    justify-self: end;
    background: #6856e8;
    color: #fff;
}

.chat-form {
    display: flex;
    gap: 10px;
    border-top: 1px solid #edf0f4;
    padding-top: 20px;
}

.chat-form input {
    flex: 1;
}
```

```
border: 1px solid #dfe4eb;
border-radius: 9px;
outline: 0;
padding: 11px 13px;
}

.modal-backdrop {
  position: fixed;
  z-index: 10;
  inset: 0;
  background: rgba(12, 18, 31, 0.53);
  display: grid;
  place-items: center;
  padding: 20px;
}

.modal {
  width: min(510px, 100%);
  max-height: 90vh;
  overflow: auto;
  padding: 25px;
  border-radius: 15px;
  background: #fff;
  box-shadow: 0 25px 70px rgba(0, 0, 0, 0.22);
}

.modal-heading {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  margin-bottom: 26px;
}

.modal-heading h2 {
  margin: 7px 0 0;
}

.modal label {
  display: block;
  color: #475365;
  font-size: 12px;
  font-weight: 700;
  margin-bottom: 17px;
}

.modal input,
.modal textarea {
  width: 100%;
  display: block;
  margin-top: 7px;
  border: 1px solid #dfe4eb;
  border-radius: 8px;
  outline: 0;
  padding: 11px 12px;
  color: #18212f;
  resize: vertical;
}
```

```
}

.modal input:focus,
.modal textarea:focus {
  border-color: #8174e2;
  box-shadow: 0 0 0 3px #eeeaff;
}

.modal small {
  display: block;
  color: #929cab;
  font-weight: 400;
  margin-top: 5px;
}

.modal-actions {
  display: flex;
  justify-content: flex-end;
  gap: 9px;
  margin-top: 25px;
}

@media (max-width: 1050px) {
  .stat-grid {
    grid-template-columns: repeat(2, 1fr);
  }

  .project-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (max-width: 760px) {
  .sidebar {
    display: none;
  }

  .main-content {
    width: 100%;
  }

  .topbar {
    padding: 0 18px;
  }

  .mobile-menu {
    display: grid;
    place-items: center;
    border: 0;
    background: transparent;
    color: #647084;
  }

  .search-box {
    width: 100%;
  }
}
```

```
.profile {
  display: none;
}

.page {
  padding: 28px 18px 50px;
}

.page-heading,
.workspace-heading,
.section-heading {
  align-items: flex-start;
  flex-direction: column;
}

.stat-grid,
.project-grid,
.workspace-grid,
.gap-grid {
  grid-template-columns: 1fr;
}

.stat-grid {
  margin: 28px 0 35px;
}

.workspace-heading .primary-button {
  width: 100%;
}

.tabs {
  gap: 16px;
  overflow-x: auto;
}

.tab {
  white-space: nowrap;
}

.paper-row {
  align-items: flex-start;
  flex-wrap: wrap;
}

.paper-info {
  width: calc(100% - 55px);
}

.paper-row .status-badge {
  margin-left: 53px;
}

.notice {
  margin: 16px 18px -3px;
```

```
}  
}
```

6. Run the application

Open two terminals.

Terminal 1: backend

```
cd researchpilot/server  
npm run dev
```

You should see:

```
ResearchPilot API running at http://localhost:5000
```

Terminal 2: frontend

```
cd researchpilot/client  
npm run dev
```

Open the local address shown by Vite, usually:

```
http://localhost:5173
```

Check the backend directly at:

```
http://localhost:5000/api/health
```

It should return:

```
{  
  "success": true,  
  "message": "ResearchPilot API is running"  
}
```

7. What already works

This version supports:

- Responsive dashboard.
- Project creation.
- Project search.
- Project deletion.

- Paper upload.
- PDF validation.
- PDF text extraction.
- Paper metadata.
- Structured summary interface.
- Gemini integration when an API key is configured.
- Local demo analysis without a Gemini key.
- Research-gap interface.
- Research chat interface foundation.
- Project pipeline status.
- Empty states and error messages.

8. Important next backend upgrade

The current backend uses in-memory arrays for fast local testing:

```
const projects = [];  
const papers = [];  
const analyses = [];
```

Before deployment, replace these with:

- Firebase Authentication.
- Firestore.
- Firebase Storage.
- Firebase security rules.

The current version is a working development foundation, not a production data-storage system. Restarting the backend clears its in-memory data. The next implementation stage should connect Firebase Authentication and Firestore so each user has private, persistent projects and papers.

1. ResearchPilot-AI-Mini-Project-Proposal.docx